

An Exact Snoop Filter for Seamless Cache Coherence in FPGA-Based Multi-Core RISC-V

Anonymous submission

Abstract—Attaching a private, snoop-coherent L1 data cache to an unmodified RISC-V core is an attractive way to build FPGA multi-core systems, and a recently proposed design does so with coherence handled entirely in hardware. We reproduce that design in open-source RTL and measure where its time actually goes. Across four benchmark kernels the cluster issues 1,048,821 invalidation broadcasts and not one of them invalidates a line in another cache. Every kernel partitions its data per processing element, so every written line is private, yet each write still pays a cluster-wide arbitration and a handshake that waits on every other cache. That accounts for between 6 and 33% of a core’s execution time, and on a streaming kernel it makes the cache extension slower than having no cache at all. We add a snoop filter that keeps an exact mirror of the caches’ tag arrays and suppresses a broadcast when no other cache holds the line or is fetching it – a distinction that is not cosmetic, since a mirror of committed tags alone loses invalidations. It costs 1,064 LUTs and 516 flip-flops, preserves coherence under a version-monotonicity checker, and is worth a geometric mean $1.12\times$, up to $1.22\times$ on matrix multiplication. It also converts the streaming kernel from $0.98\times$ to $1.10\times$ against an uncached baseline, at the original design’s own 60 MHz operating point and for 66 ps of timing slack.

Index Terms—RISC-V, cache coherence, snoop filter, FPGA, multi-core

I. INTRODUCTION

Multi-core systems built on FPGAs increasingly rely on private data caches to keep soft processor cores fed from a shared memory whose latency they cannot otherwise hide. Once each core caches shared data, coherence must be maintained, and its cost is charged against the very performance the caches were added to provide – paid not only in the area of the tag and status structures, but in the serialisation any cluster-wide coherence action imposes on the cores waiting for it.

Accordingly, many hardware and software coherence schemes have been proposed for scalable architectures [2]–[4], and the RISC-V ecosystem offers coherent fabrics at scale [5]–[7]. Adopting one, however, means adopting its interconnect and usually modifying the core, which for a multi-core assembled from existing verified soft cores is precisely the cost the designer is trying to avoid.

A seamless cache extension avoids it. Kamaleldin *et al.* [1] attach a private write-through L1 data cache unit (DCU) to an unmodified RISC-V core through its ordinary memory interface and maintain coherence with a snooping invalidation bus, in hardware and invisibly to software. What such a design has not been asked is how much of the coherence traffic it generates is *necessary* – a question answerable only by instrumenting the bus and counting.

In this work, we reproduce that design in open-source SystemVerilog in order to measure it. Between 6 and 33% of a core’s time is spent waiting on the coherence bus, and across four benchmark kernels the cluster sends 1,048,821 invalidation broadcasts of which not one invalidates a line anywhere. The cause is structural: the kernels partition their arrays per processing element, so almost every written line is private and the bus has no way to know it. We therefore propose an exact snoop filter that answers the question before the bus arbitrates. Our contributions are listed as follows:

- A measurement of how much of this design’s coherence traffic is necessary. On its own benchmark kernels, none of it is, and we quantify what that costs (Section IV).
- An exact snoop filter: a structural mirror of the caches’ tag arrays that lets the bus suppress a broadcast no cache needs. Exactness has to be over the right property: a mirror of committed tags alone is unsound, because a cache with a refill in flight holds nothing yet and must still be told (Section V).
- A cost model of the bus protocol that derives the filter’s benefit from the grant condition, and is checked against the measured saving rather than fitted to it (Section V-D).
- An evaluation in cycles, in coherence correctness, and in place-and-routed FPGA cost, including the frequency the mechanism costs and the operating point at which it pays (Section VI), and three mechanisms rejected before this one, including one that was incorrect while passing every functional test it had (Section VII).

II. RELATED WORK

Filtering snoops is not a new idea, and the value here is not the concept but where the filter sits, what it is exact about, and what it is measured against.

JETTY [2] places a small cache-like structure between the bus and each processor’s L2, filtering snoops that would miss locally so that the energy of a tag lookup is not spent. Its target is energy, its position is the receiving side, and it is approximate: a small JETTY filters most useless snoops, not all. Coarse-grain coherence tracking [3] and RegionScout [4] raise the granularity instead, tracking sharing over regions of hundreds or thousands of bytes so that a region known to be unshared needs no broadcast at all, trading precision for a much smaller structure.

Our setting inverts the cost that matters. On a snooping bus in a soft multi-core, the expense of an unnecessary invalidation is not the energy of the remote tag lookups but the *serialisation*: one grant per cycle across the cluster, held until

every other cache is ready to accept the broadcast. The filter therefore belongs at the source, deciding whether to arbitrate at all, rather than at each receiver deciding whether to look. And because the design is small – four caches of 128 lines each – an exact mirror is affordable, at 1,064 LUTs (Section VI), which removes the need to trade precision for size and reduces the correctness argument to a sentence.

The RISC-V ecosystem offers coherent fabrics at a range of scales – TileLink [5], OpenPiton [6] and CIFER [7] – but these are complete memory systems. The design studied here is deliberately a different point: an extension attached to an unmodified core through its existing memory port, which is what makes it attractive on an FPGA and also what confines coherence to a single shared bus.

Finally, JETTY’s premise is that many snoops find no copy elsewhere. On the benchmark kernels of the design studied here, *none* does (Section IV); the difference between “many” and “all” is a property of data-parallel workloads on a private-cache cluster, and it is what makes an exact filter worth its area here.

III. THE DESIGN UNDER STUDY

The DCU is a private, write-through, no-allocate L1 data cache sitting between an in-order RISC-V core and the memory system. Because writes go through to memory, shared memory always holds the current value and no dirty-line write-back or ownership state is needed. Coherence is maintained by an invalidation bus: a core write reaches the bus as an invalidation request, and on grant the address is broadcast to every other DCU, which drops any matching line.

Two properties of that bus matter here. Plain read requests are not arbitrated against each other – multiple readers proceed in the same cycle, which is the multiple-readers half of the MRSW invariant – but a read is withheld while another core has an invalidation outstanding for the same line, which is the single-writer half. Invalidation *are* arbitrated: at most one is granted per cycle across the cluster, and the grant is held back until every other DCU can accept the broadcast, so that no invalidation is ever lost.

Fig. 1 shows the system and where the filter sits in it. Our reproduction targets the stated configuration: four processing elements, each an unmodified CV32E40P with a private 2 KiB two-way DCU of 16-byte lines, meeting shared memories through an AXI4 crossbar. A non-coherent baseline is built from the same sources with one parameter changed, so any comparison between them is attributable to the cache subsystem alone.

IV. WHERE THE TIME GOES

A. The coherence bus is a large fraction of runtime

The DCU separates a core access into the places it can wait. One of those is stage 1 waiting for a snoopy-bus grant, as a fraction of the cycles a core spent with an access in the DCU at all.

At a memory latency of two the share is 14.3% on memcpy, 15.8% on matmul, 6.5% on conv2d and 5.8% on FFT; at

twenty it is 33.0, 32.9, 19.0 and 12.2%. Between a twentieth and a third of a core’s time goes to arbitrating for a resource that grants one request per cycle across the whole cluster, and the share grows with memory latency because each grant is held longer.

B. It is not readers waiting on writers

The obvious reading of that is contention between readers and writers – the single-writer screening doing its job. We instrumented the bus to count it, and across all four kernels a read is *never* withheld: not once. Whatever the stall is, it is not the MRSW screening.

By elimination it is the invalidations, which are the only arbitrated traffic left.

C. None of the invalidations are necessary

Each DCU therefore counts the broadcasts it receives and how many find a line to clear.

The counts are 823,338 broadcasts on matmul, 139,182 on conv2d, 61,683 on FFT and 24,618 on memcpy – over a million in total, and the useful column is zero in every one of the four. Each one paid a cluster-wide arbitration and a four-way handshake to inform nobody of anything.

A result this uniform deserves suspicion of the instrument rather than of the design, so the counter is controlled. Our coherence testbench shares deliberately, with a producer/consumer ping-pong and a randomised four-core stress over a small window; on that workload the same counter reports 345 useful broadcasts of 921. The counter can see a useful invalidation. The zeroes are a property of the workloads.

They are a property worth stating plainly: these kernels are data-parallel and partition their data per core, so essentially every written line is private. That is not a weakness of the benchmark selection, because these are the kernels the original evaluation itself uses.

V. AN EXACT SNOOP FILTER

A. Structure

The bus needs to answer one question before granting an invalidation: is there another cache that has to hear it? A mirror of the caches’ tag and valid arrays answers it, held in the bus and updated by the same two events that change the caches themselves – a refill installs a tag into a way, an invalidation clears one. Each DCU exports those events on a small port driven from the same signals that write its own arrays.

Exactness is therefore structural rather than argued. The mirror is written from the same signals, on the same clock edge, under the same conditions as the state it mirrors, so it cannot drift from it.

We rejected a hashed filter deliberately. A Bloom-style structure may only over-approximate, since claiming a line is held when it is not is safe and the converse loses an invalidation; an insert-only one saturates, and matmul touches far more lines than an affordable bitmap has bits, while removing from one requires knowing when a line leaves a cache – exactly what a mirror already has.

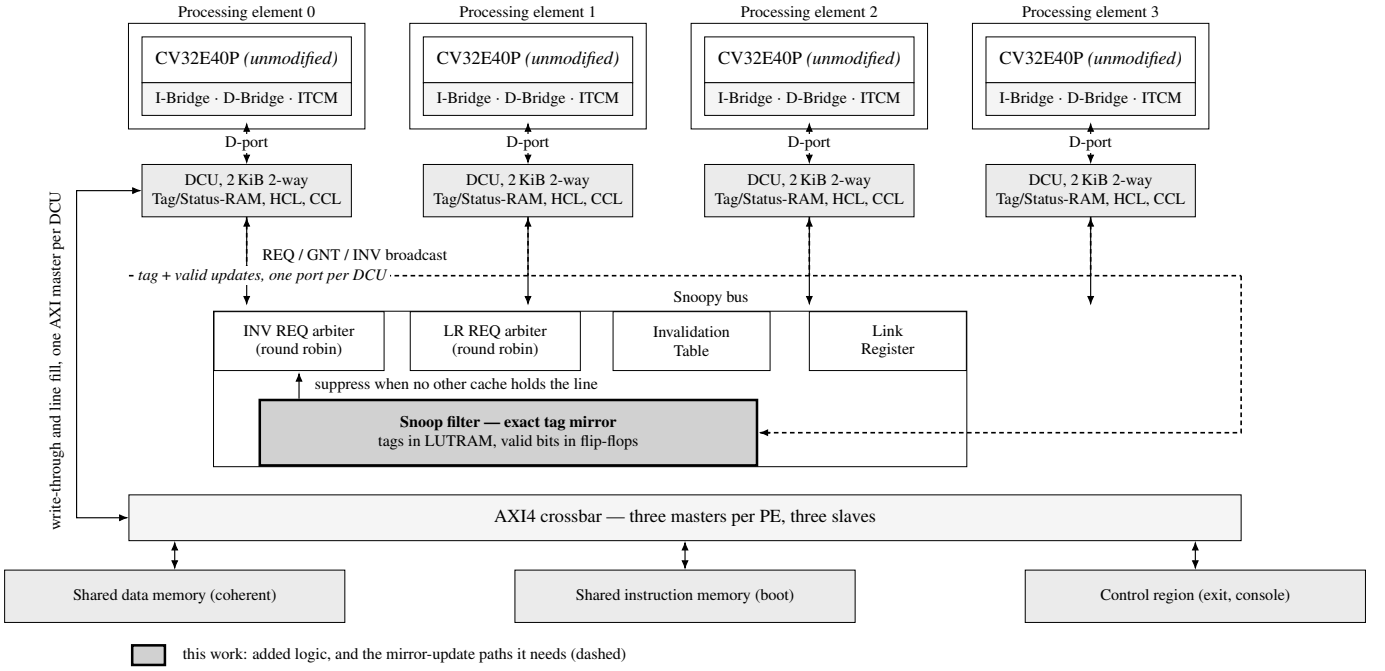


Fig. 1. The system, and what this work adds to it. Four unmodified cores sit over their private data cache units, which reach shared memory through an AXI4 crossbar and maintain coherence through a single snoop bus. The snoop filter holds a mirror of every cache’s tag and valid arrays, kept current by one update port per DCU, and answers the invalidation arbiter’s one question: is there anybody to tell?

B. Exact about the wrong property

A mirror of the tag arrays is exact about which lines each cache *holds*, and that turns out not to be the property the bus needs. A DCU between issuing its memory read and refilling the victim way holds nothing for the line it is fetching, so it does not appear in the mirror – and it is precisely the cache that must receive the invalidation, because receiving one in that window is what cancels its refill. Suppress the broadcast and it installs a line that was invalidated while in flight.

The property is *residency or imminent residency*. Each DCU therefore exports its refill window alongside its mirror updates, and the filter answers over the union. The window opens when the cache issues the fetch and closes on the cycle the refill lands, which is the same cycle the mirror learns the tag: the two halves abut exactly, so the line is covered in every cycle by one or the other.

This is not a defensive addition. Counting the granted invalidations for which no other cache held the line but one was fetching it – each a broadcast a committed-tag mirror would have wrongly suppressed – our coherence testbench reports 20 of 2,241 over ten randomised interleavings, at least one in each. The window is narrow enough that the design passed 3,139 checks before the counter existed, which is how a race of this shape usually survives a test suite.

C. Why suppression is safe

When the answer is that no other cache holds the line or is fetching it, a broadcast could clear nothing anywhere, and not sending it is unobservable. That is the whole argument, and

it is short because the structure is exact. A conservative filter would need an argument about the direction of its error; this one needs only that it asks about the right state.

Everything else is untouched. Invalidations that *are* needed are still arbitrated, broadcast and acknowledged exactly as before; the single-writer screening on reads is unchanged; and the write-through lock that stops a remote read fetching a pre-write value is unchanged, because it protects against a value in memory rather than against a cached copy.

D. Why suppression pays

Safety says the broadcast may be dropped; this says what dropping it is worth, from the protocol rather than from the measurement.

Let core i issue a write w that reaches stage 1 of its DCU at cycle $a(w)$. The original protocol grants it at the first cycle in which the arbiter selects it *and* every other DCU is ready to accept an invalidation,

$$g(w) = \min \left\{ t \geq a(w) : \text{arb}(t) = i \wedge \bigwedge_{j \neq i} r_j(t) \right\}. \quad (1)$$

A DCU is ready, $r_j(t) = 1$, only while its stage 1 is not already holding an invalidation, and an invalidation leaves stage 1 only when stage 2 is free – which it is not for the full memory latency of any miss or write-through in flight. On grant, one invalidation is injected into stage 1 of each of the other $P-1$ DCUs, where it takes precedence over the local core: a request already there is displaced to a holding slot, one the core wanted

to issue is refused, and either way the local pipeline is shifted by one cycle. The cost of w to the cluster is therefore

$$C(w) = (g(w) - a(w)) + \sum_{j \neq i} \rho_j(g(w)), \quad (2)$$

with $\rho_j(t) \in \{0, 1\}$ one when DCU j had a request to shift and zero when it was idle.

Two things follow. The conjunction in (1) couples every pipeline in the cluster: a store on core i waits for a load miss on core j to complete, whether or not the two touch a common line. That is why the stall share of Section IV grows with memory latency – the conjunction is held false for the length of each miss. And neither term of (2) depends on what any cache holds; the protocol charges $C(w)$ for every write alike.

The filter makes it depend. With the sharer set $S(w)$ read from the mirror, the grant condition becomes $\text{arb}(t) = i \wedge (S(w) = \emptyset \vee \bigwedge_{j \neq i} r_j(t))$ and invalidations are injected only when $S(w) \neq \emptyset$. For a write with no sharers both terms of (2) vanish, leaving the arbitration wait alone, which is non-zero only in a cycle where two cores present writes at once. Summed over a run in which every write has an empty sharer set – which Section IV measured to be the case on all four kernels – the saving is the whole coherence overhead less arbitration collisions: exactly the oracle, which is why the filter reaches its cycle for cycle rather than approximately.

Both terms are observable, so the model can be checked rather than fitted. The sender term is the snoop-stall counter; the receiver term is at most one cycle per invalidation delivered; so the two bracket the saving. Per DCU, at a memory latency of two, the measured saving sits inside that bracket on every kernel: $\text{matmul } 120,539 \leq 243,100 \leq 326,373$ cycles, $\text{conv2d } 5,185 \leq 13,795 \leq 39,980$, $\text{FFT } 892 \leq 4,865 \leq 16,312$, $\text{memcpy } 1,792 \leq 2,302 \leq 7,946$. The share of the receiver bound that materialises – 0.60, 0.25, 0.26 and 0.08 – is highest on matmul, whose pipeline is busy with in-cache reads when the invalidations land, and lowest on memcpy, where the displaced request was waiting on a write-through anyway.

E. Implementation

The mirror is one array per (core, way) pair, each with a single write port and a single read port, addressed by set index – eight arrays at the geometry studied here. Tags are held outside any reset, since a tag is only compared when its valid bit says it means something, which lets them map to distributed RAM. Valid bits stay in flip-flops, where they can be reset and read eight at a time, at 512 bits rather than 11,264. This is the same division the DCU makes internally for the same reason.

That structure matters more than it might appear: a first implementation held tags and valid bits together under an asynchronous reset and read them combinatorially, which Vivado mapped entirely to flip-flops – 11,776 of them, and 3,202 LUTs, against 512 and 408 for the structure above.

TABLE I
EXECUTION CYCLES OF THE SLOWEST PROCESSING ELEMENT, BEHIND FIG. 2. THE LAST TWO COLUMNS ARE THE REDUCTION AGAINST NO DATA CACHE, FOR THE ORIGINAL DESIGN AND WITH THE FILTER ADDED.

Kernel	Size	Execution cycles			Reduction	
		No cache	Cache	+Filter	Cache	+Filter
matmul	32 ²	277,621	171,142	140,223	38.4%	49.5%
	64 ²	2,191,981	1,337,850	1,094,750	39.0%	50.1%
	128 ²	16,899,384	9,461,615	8,118,893	44.0%	52.0%
conv2d	32 ²	73,268	38,120	35,526	48.0%	51.5%
	64 ²	311,454	168,177	154,382	46.0%	50.4%
	128 ²	1,277,966	683,079	625,110	46.5%	51.1%
FFT	256	113,092	85,093	80,228	24.8%	29.1%
	512	245,292	244,338	225,852	0.4%	7.9%
	1024	550,489	574,366	541,289	−4.3%	1.7%
FFT, blocked	256	121,213	93,087	88,604	23.2%	26.9%
	512	269,224	238,776	219,017	11.3%	18.6%
	1024	584,916	523,833	478,210	10.4%	18.2%

VI. EVALUATION

A. Cycles

Fig. 2 places the extension on the original’s own axes: execution time per kernel against problem size, uncached against cached, with our filter added as a third bar the original has no counterpart to. Table I gives the cycle counts behind it.

Matrix multiplication and convolution reproduce the $\approx 40\%$ the original reports for both, and neither decays with problem size. That is not luck. What must stay resident is not the data set but the *reuse footprint*, the bytes a core revisits before moving on: one row of B , $4N$ bytes, for matrix multiplication, and the three-row sliding window, $12N$ bytes, for convolution. At the largest size those are 512 B and 1536 B, both inside a 2 KiB DCU, so the benefit does not move.

FFT is where that stops holding. The original reports a uniform $\approx 23\%$; we match it at $N=256$ and then watch the benefit vanish and reverse, until at $N=1024$ the cache makes the kernel *slower* than having none. The footprint argument predicts exactly this: FFT revisits N complex points across its stages, $8N$ bytes, which is 2 KiB at $N=256$ and 8 KiB at $N=1024$. Only the first fits.

That the cause is capacity and not conflict can be shown by moving one variable at a time. At $N=512$, doubling associativity at 2 KiB recovers 3.8%; quadrupling capacity recovers 24.5%, and 16 KiB recovers nothing beyond 8 KiB – saturation once the footprint fits. Given a DCU that holds the footprint, FFT reproduces the original at every size, 24.8, 24.8 and 24.5% at 8 KiB; at the 2 KiB its text states, it does not. We read that as a configuration discrepancy rather than a contradiction: its result implies a larger cache or a smaller working set than described, and the text does not let a reader tell which.

The collapse is a property of the kernel’s access order, not the hardware, and the fourth panel of Fig. 2 undoes it in software. A radix-2 butterfly at stage s pairs elements 2^s apart;

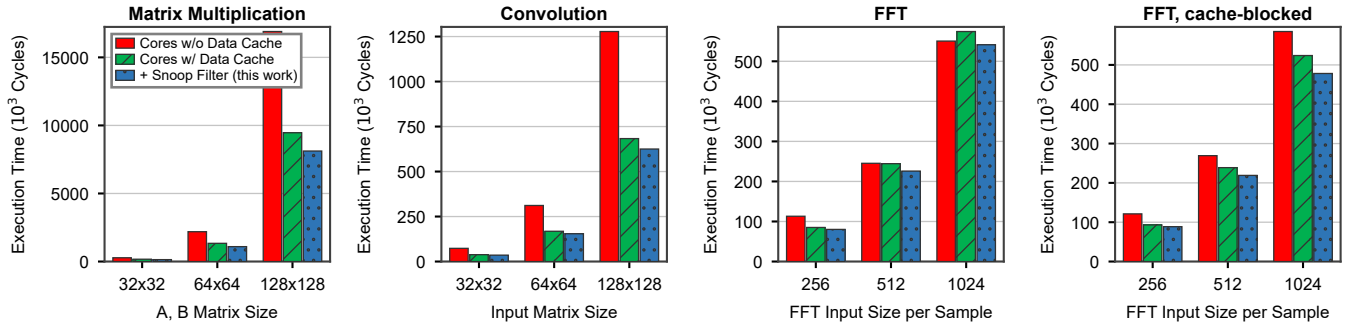


Fig. 2. Execution time against problem size, without a data cache, with one, and with the snoop filter of this work. The first three panels are on the original’s own axes, so their first two bars are directly comparable with it. The fourth is the same FFT with its butterfly stages reordered so the working set stays inside the cache – bit-identical output – which is our answer to the third. Table I gives the values.

once 2^s is a multiple of a block size B the late stages never mix blocks, so they are B independent transforms on strided columns. Running the early stages block by block and the late ones column by column keeps the working set inside the DCU throughout, with every butterfly unchanged – the output is bit-identical, and passing the plain kernel’s own checksum is the proof. At $N=1024$ the extension goes from -4.3% to $+10.4\%$, and with the filter to 18.2% ; at $N=256$, where everything already fitted, the extra loop structure costs about 6% .

None of it weakens the filter, which is orthogonal: it removes coherence traffic, not misses, and its share is steady at seven to eight points across both FFT panels.

Against the unfiltered design the filter is worth $1.222\times$ on matrix multiplication, $1.123\times$ on memcpy, $1.089\times$ on convolution and $1.061\times$ on FFT: a geometric mean of $1.122\times$. The useful comparison is not that mean but an oracle that suppresses every broadcast unconditionally, which is sound to simulate only because none of them are needed, and which bounds what any filter could return.

The filter reaches the oracle on every kernel, cycle for cycle, as Section V-D says it must when every sharer set is empty. The snoop-stall bucket confirms the mechanism: on conv2d it falls from 20,848 cycles to 109, on memcpy from 7,168 to 1. FFT falls only from 6,521 to 2,955, because load-reserved and store-conditional still pass the arbiter whatever the broadcasts do – a residual no filter can remove.

B. The extension stops being harmful

The reproduction’s most awkward finding was that on memcpy at low memory latency, the cache extension is *slower* than having no cache at all: $0.975\times$ the non-coherent baseline. With the filter it runs at $1.095\times$. The pathology is not intrinsic to attaching a coherent cache; it is the cost of announcing private writes to a cluster that does not care.

Fig. 3 shows it holds at every transfer size and locates where it stops. Over 4 to 32 KiB the extension is 2.5 to 7.6% slower at every size, and the filter turns that into 8.2 to 9.0% faster. Raising the memory latency restores the original’s result: 34% at eight and 56% at twenty, against its reported $\approx 50\%$. A

streaming kernel offers a cache no reuse, only latency to hide; when there is little of that, the coherence overhead is all that remains.

C. Coherence

The four kernels never exercise the filter’s must-broadcast path, so they cannot establish that it is safe. Our coherence testbench does: it shares deliberately, half of its invalidations are useful, and with the filter enabled it passes 3,112 checks, including a version-monotonicity invariant – a core that has observed version k of a location never afterwards observes one below k , which is what a lost invalidation causes – and a final quiesce that re-reads the whole window against memory. The filter suppressed 282 of 921 broadcasts and delivered every one that mattered.

D. FPGA cost

Table II gives the breakdown, from a non-project Vivado 2025.1 flow on a Zynq UltraScale+ XCZU7EV. The filter costs 1,064 LUTs and 516 flip-flops over the unfiltered design, under 4% of it, and no block RAM or DSP at all. Both configurations meet every constraint with no failing endpoints, at $+1.199$ ns and $+1.265$ ns of slack respectively – so at this operating point the filter costs essentially no slack, 66 ps on a 16.667 ns period.

Both figures come from one source tree with one parameter changed, implemented in the same batch, which is the only way a slack comparison on this design means anything: an earlier pair of runs weeks apart put the same two configurations 1.7 ns apart in the opposite direction, from netlists differing by fourteen LUTs. We quote slack only within a batch.

The breakdown confirms the structure went in as intended: half of the filter’s 408 LUTs are distributed RAM holding the mirror, and its 512 flip-flops are exactly the valid bits, four cores by 64 sets by two ways.

E. The frequency it costs

At 60 MHz the filter costs 66 ps; at a 75 MHz target an earlier pair of builds had the reference meeting timing and the filtered design missing by 2.586 ns. The critical path

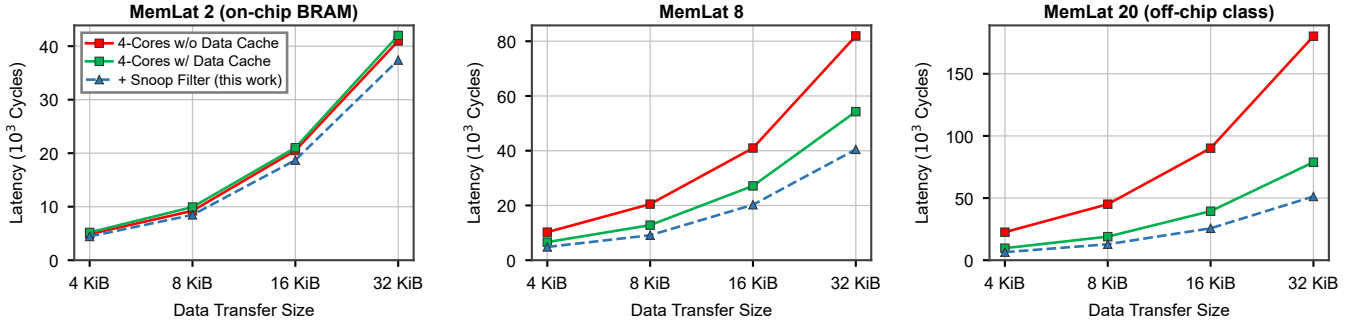


Fig. 3. Memory-copy latency against transfer size, on the original’s axes, at three memory latencies. At two the extension sits *above* the uncached line at every size and only the filter brings it below; the original’s $\approx 50\%$ appears between eight and twenty.

TABLE II

POST-IMPLEMENTATION RESOURCES, XCZU7EV AT 60 MHz, FILTERED BUILD. SUB-ROWS MAY SUM SLIGHTLY ABOVE THEIR PARENT, BECAUSE A LUT SHARED ACROSS A HIERARCHY BOUNDARY IS COUNTED AGAINST BOTH CHILDREN BUT ONLY ONCE AGAINST THE PARENT. THE FIGURES BELOW THE RULE ARE THE TOOL’S OWN, NOT COLUMN SUMS.

Module	LUTs	FFs	BRAMs	DSPs
Processing element ($\times 4$)	17,833	9,948	32	20
RISC-V core, CV32E40P	16,929	8,404	0	20
ITCM + Instr.-Bridge	252	36	32	0
Data-Bridge	319	32	0	0
AXI4 masters ($\times 3$ per PE)	333	1,476	0	0
Cache extension	6,540	4,628	20	0
Data Cache Unit ($\times 4$)	5,816	3,998	20	0
controller + HCL	2,860	2,846	0	0
tag RAM	1,677	96	4	0
data RAM	1,289	1,056	16	0
Snoopy bus	728	630	0	0
round-robin arbiter	251	2	0	0
Invalidation Table	69	116	0	0
snoo filter (this work)	408	512	0	0
AXI4 crossbar	2,125	279	0	0
Shared data memory, 256 KiB	724	150	64	0
Shared instr. memory, 32 KiB	506	86	8	0
Control region	113	259	0	0
Whole device, as reported by the tool	27,845	15,670	124	20
Available	230,400	460,800	312	1,728
Utilisation (%)	12.09	3.40	39.74	1.16

is 86% routing and spans the die from a DCU’s registered request address: the decision to broadcast is a cluster-spanning combinational path however the mirror is built. A 3.9 ns gap across two runs is not separable from this design’s 1.7 ns of batch-to-batch variation, so we treat the 75 MHz result as indicative and evaluate at 60 MHz, the frequency the original itself runs at, where the filter’s $1.122\times$ in cycles is a $1.122\times$ in wall time.

VII. WHAT DID NOT WORK

Three mechanisms preceded this one, each rejected by a measurement.

A hit-speculative bypass, in which only a miss arbitrates, was useless – plain reads were never arbitrated, so it measured slower – and incorrect: a read that hits while a remote

invalidation for the line is outstanding returns stale data, the single-writer half of MRSW. It passed all eleven functional checks, which is the point worth reporting.

Two restructurings of the mirror: neither cured the frequency loss (Section VI).

Directed invalidation, the obvious next step, does not pay. The mirror knows not merely *whether* another cache holds the line but *which*, so the bus can restrict both the broadcast and the conjunction of (1) to $S(w)$: an exact multicast with a partial barrier, which only an exact filter may attempt, since a missing sharer bit under multicast loses an invalidation rather than costing a redundant one. On a halo-exchange stencil whose band boundaries give a sharer set of one of three, it delivers 168 invalidations, all useful, and is worth 0.1 to 0.3% at every memory latency tried. Once suppression has removed every write with $S(w) = \emptyset$, what remains of (2) is irreducible: somebody has to be told and the writer has to wait.

VIII. THREATS TO VALIDITY

The central measurement is that none of the invalidation traffic is useful, and that is a property of workloads as much as of the design: data-parallel kernels that partition per core have almost nothing to share. We claim only that on the kernels the original evaluation itself uses, this design’s coherence traffic is entirely avoidable. The halo-exchange stencil of Section VII has genuine sharing, and the filter is still worth $1.11\times$ there at a memory latency of twenty, because the interior of each band is private even when its boundary is not.

The evaluation is in cycles from RTL simulation and in post-place-and-route figures; no bitstream has been run on hardware, and the original targets an XCVU9P rather than our XCZU7EV, so absolute resource figures are directional. Power is not reported: Vivado’s vectorless estimate models the design as idle.

IX. CONCLUSION

A seamless coherent cache extension spends 6 to 33% of each core’s time on a protocol that, on its own benchmark kernels, accomplishes nothing. A mirror of the caches’ tag arrays that is exact over residency *and* imminent residency suppresses it for 1,064 LUTs and is worth $1.122\times$.

REFERENCES

- [1] A. Kamaleldin, M. Nickel, S. Wu and D. Göhringer, “Seamless Cache Extension for FPGA-based Multi-Core RISC-V SoC,” in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, 2024.
- [2] A. Moshovos, G. Memik, B. Falsafi and A. Choudhary, “JETTY: Filtering Snoops for Reduced Energy Consumption in SMP Servers,” in *Proc. 7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 2001.
- [3] J. F. Cantin, M. H. Lipasti and J. E. Smith, “Improving Multiprocessor Performance with Coarse-Grain Coherence Tracking,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 246–257.
- [4] A. Moshovos, “RegionScout: Exploiting Coarse Grain Sharing in Snoop-Based Coherence,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 234–245.
- [5] W. Terpstra, “TileLink: A Free and Open-Source, High-Performance Scalable Cache-Coherent Fabric Designed for RISC-V,” in *RISC-V Workshop*, 2017.
- [6] J. Balkind *et al.*, “OpenPiton+Ariane: The First Open-Source, SMP Linux-booting RISC-V System Scaling from One to Many Cores,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.
- [7] A. Li *et al.*, “CIFER: A Cache-Coherent 12nm 16mm² SoC with Four 64-Bit RISC-V Application Cores, 18 32-Bit RISC-V Compute Cores, and a 1541 LUT6/mm² Synthesizable eFPGA,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023.